



PROYECTO INTEGRADOR

ESTADISTICA INFERENCIAL II

UNIDAD

1

REGRESIÓN LINEAL SIMPLE Y CORRELACIÓN

JAZMIN GUADALUPE ROJAS VARGAS
OSCAR SAID VALENCIANO GALLEGOS

RUBEN DARIO DIAZ VITAL
PALOMA GUADALUPE SERVIN

BELTRAN

N.C. 24151211

N.C. 24150385

N.C. 23150489

N.C. 24150680

03/07/2026

¿Cual es el equipo con mayor posibilidad de ganar el mundial?

Requisitos

La variable dependiente seria el numero de goles metidos en cada partido (X) mientras que la variable independiente serian los tiros totales y tiros de esquina (Y) ya que de este tipo de acciones se sabe si habra o no goles en un partido.

Se espera tener una relacion linal ya que cuando hay cambios en la variable (X) provoca cambios proporcionales y constantes en la variable (Y).

Para llevar a cabo la recoleccion de datos tuvimos que esperar a que fueran todos los partidos de la fase de grupos del mundial. Una vez acabados nosotros por medio de la aplicacion OneFootball revisamos las estadisticas y datos necesarios para nuestro problema y terminado analizamos la cantidad de tiros totales y tiros de esquina para definir como esto afecta al numero de goles anotados

Introduccion

Saber si se puede predecir cuales son los posibles o posible candidato ganador a la copa mundial mediante el analisis de goles durante los partidos de la fase de grupos, es importante estudiarlo para poder tener objetividad y claridad para saber como es que muy posiblemente quede el campeonato, la hipotesis incial es que los equipos que jueguen los partidos con mas tiros realizados son los que mas posibilidades tienen de llegar mas lejos en el mundial y basicamente el objetivo es saber con mayor exactitud quie seria el ganador de la copa mundial 2026

INTRODUCCION

¿Cuál es el problema que desean resolver?

¿Por qué es importante estudiarlo?

¿Cuál es la hipótesis o expectativa inicial?

¿Cuál es el objetivo del proyecto?

METODOLOGIA

2. Metodología

- ¿Cómo se obtuvieron los datos?

con una aplicación de fútbol que proporcionaba los datos necesarios

- ¿Qué población representa la muestra?

- ¿Cuál fue el tamaño de la muestra?

72 datos de mismos y diferentes días

- ¿Cuál es la variable independiente?

Los goles totales

- ¿Cuál es la variable dependiente?

Tiro totales

- ¿Cómo se midieron ambas variables?

- ¿Qué procedimiento estadístico se aplicó?

- ¿Por qué la regresión lineal simple es adecuada para este problema?

```
In [ ]: import pandas as pd

df =
pd.read_csv("https://raw.githubusercontent.com/JazVargaskS20/EstadisticaVerano2026/refs/heads/main/StudentPerformance/PROYECTO FUT.csv")

df
```

	D'a_del_partido	Tiros_de_esquina	Tiros_totales	Goles_totales
0	11-jun	6	19	2
1	11-jun	9	22	3
2	12-jun	13	21	2
3	12-jun	4	25	5
4	13-jun	13	32	2
...	...	...	...	...
67	27-jun	5	14	3
68	27-jun	7	37	0
69	27-jun	6	22	4
70	27-jun	8	17	4
71	27-jun	3	22	6

72 rows × 4 columns

```
In [ ]: X = df['Tiros_totales']    #Variable dependiente
Y = df['Goles_totales']    #Variable independiente
```

RESULTADOS

¿Qué muestran los gráficos de dispersión?

```
In [ ]: # Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4), # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120         # resolución del gráfico
)

# Gráfico de dispersión
plt.scatter(
    X, Y,
    marker="o",      # forma: googlear "matplotlib.markers"
    color='#FD292F',  # color de los puntos
    edgecolor='#E3BDF4', # borde de los puntos
    alpha=0.9,        # transparencia: 0 es invisible y 1 es color sólido
    s=50,             # tamaño de los puntos
)

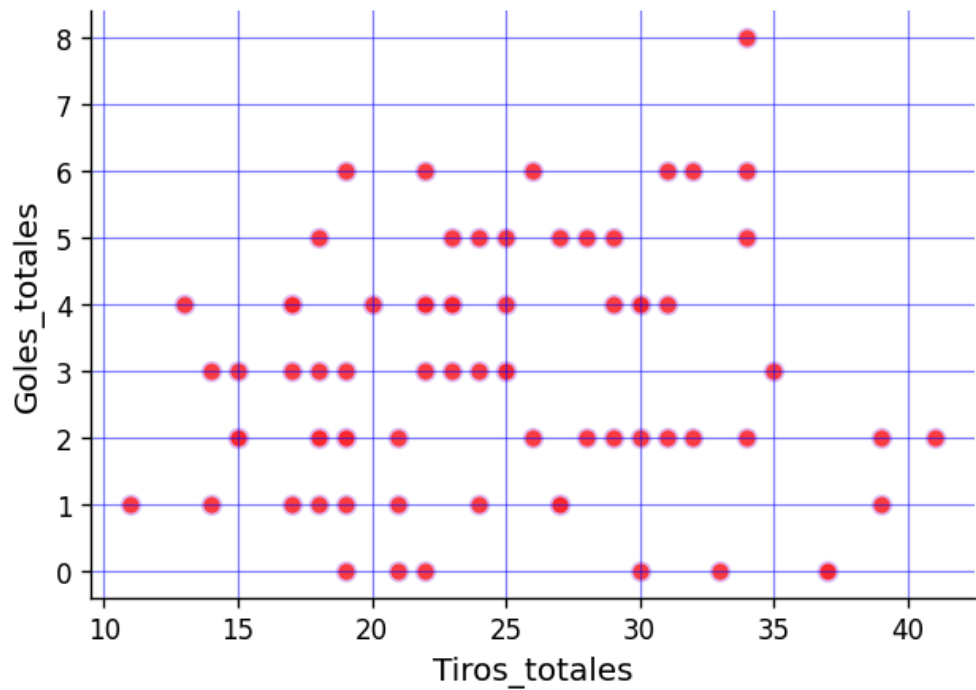
# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Tiros_totales', # etiqueta del eje x
    fontsize=12      # tamaño de fuente
)

plt.ylabel(
    'Goles_totales', # etiqueta del eje y
    fontsize=12      # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='-',
    linewidth=0.7,
    alpha=0.5,
    color="blue"
)
```



El gráfico de dispersión muestra los tiros totales (\$X\$) muestra los goles totales (\$Y\$)

Ademas que entre 15 a 35 tilos totales se pueden meter hasta 8 goles totales en un partido

2.-¿Cuál fue el valor del coeficiente de correlación y cómo se interpreta?

3.-¿Cuál fue el coeficiente de correlación y cómo se interpreta?

```
In [ ]: # Importar la función pearsonr desde scipy.stats
        from scipy.stats import pearsonr

        # Test de Pearson
        # H0: rho = 0      (No hay correlación)
        # H1: rho ≠ 0      (Sí hay correlación)
        # alpha = 0.05

        r, valor_p = pearsonr(X, Y)

        print(f"Coeficiente de correlación: {r: 0.4f}")
        print(f'valor_p: {valor_p: 0.4f}')
```

```
Coeficiente de correlación: 0.0619
valor_p: 0.6056
```

3.-¿Cuál fue el coeficiente de correlación y cómo se interpreta?

El coeficiente de correlacion es de $r=0.0619$ y esto indica que hay una linea muy baja

4.-¿Cuál fue el coeficiente de determinación y qué porcentaje de la variabilidad explica el modelo?

```
In [ ]: import statsmodels.api as sm
x_constante = sm.add_constant(X)
modelo = sm.OLS(Y, x_constante).fit()
y_calculada = modelo.predict(x_constante)
```

```
In [ ]: # Coeficiente de determinación
from sklearn.metrics import r2_score

r2 = r2_score(Y, y_calculada)

print(f"Coeficiente de determinación: {r2: 0.4%}")
```

Coeficiente de determinación: 0.3829%

5.-¿Cuál es la ecuación de regresión obtenida?

$R^2=0.3829$

6.-¿Qué significa la pendiente en el contexto del proyecto?

Una vez ajustado el modelo de regresión lineal, se obtiene un total de goles de $R^2=0.3829$. Entonces teniendo los tiros totales y el modelo ajustado, sólo podemos justificar la variabilidad en los tiros totales en un 38.29%, lo que es un poco bajo ya que tenemos que hacer muchos tiros totales en un partido para que en hayan entre 1 y 5 goles totales en un partido

7.-¿Qué indican los residuales sobre la calidad del modelo?


```
In [ ]: import pandas as pd
import statsmodels.api as sm

# Load the dataframe
df =
pd.read_csv("https://raw.githubusercontent.com/JazVargasKS20/EstadisticaVerano2026/refs/heads/main/StudentPerformance/PROYECTOOFUT.csv")

# Define X and Y variables
Y = df["Goles_totales"]      # Variable independiente
X = df["Tiros_totales"]      # Variable dependiente

# Add a constant to the independent variable
x_constante = sm.add_constant(X)

# Create and fit the OLS model
modelo = sm.OLS(Y, x_constante).fit()

# Print the model parameters
modelo.params
```

	0
const	2.530832
Tiros_totales	0.016205

dtype: float64

Indican que la constante es de \$2.53\$ y los tiros totales es de \$0.016\$

8.-¿Se cumplen los supuestos del modelo?

9.-¿Qué predicción puede realizarse utilizando la ecuación obtenida?

La ecuación de la recta es:

$$\hat{y} = 2.530832 + 0.016205X$$

Este modelo estima que la constancia que fue 2.530832 por lo que el valor esperado de goles totales en un partido es de 0 y que Goles totales es 0.016205 lo que indica que por tiro que se tire no hay tanta probabilidad que se meta en un partido

```

In [ ]: # Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4), # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120         # resolución del gráfico
)

# Gráfico de dispersión
plt.scatter(
    X, Y,
    marker="o",      # forma: googlear "matplotlib.markers"
    color='blue',    # color de los puntos
    edgecolor='#ED92E8', # borde de los puntos
    alpha=0.9,       # transparencia: 0 es invisible y 1 es color
    sólido
    s=50,            # tamaño de los puntos
)

# Gráfico de línea
plt.plot(
    X, y_calculada,
    color='red',     # color de la línea
    linewidth=1.0,   # grosor de la línea
    linestyle='-',   # estilo de línea
    label='Recta de regresión'
)

# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Tiros_totales', # etiqueta del eje x
    fontsize=12      # tamaño de fuente
)

# eje y
plt.ylabel(
    'Goles_totales', # etiqueta del eje y
    fontsize=12      # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,

```

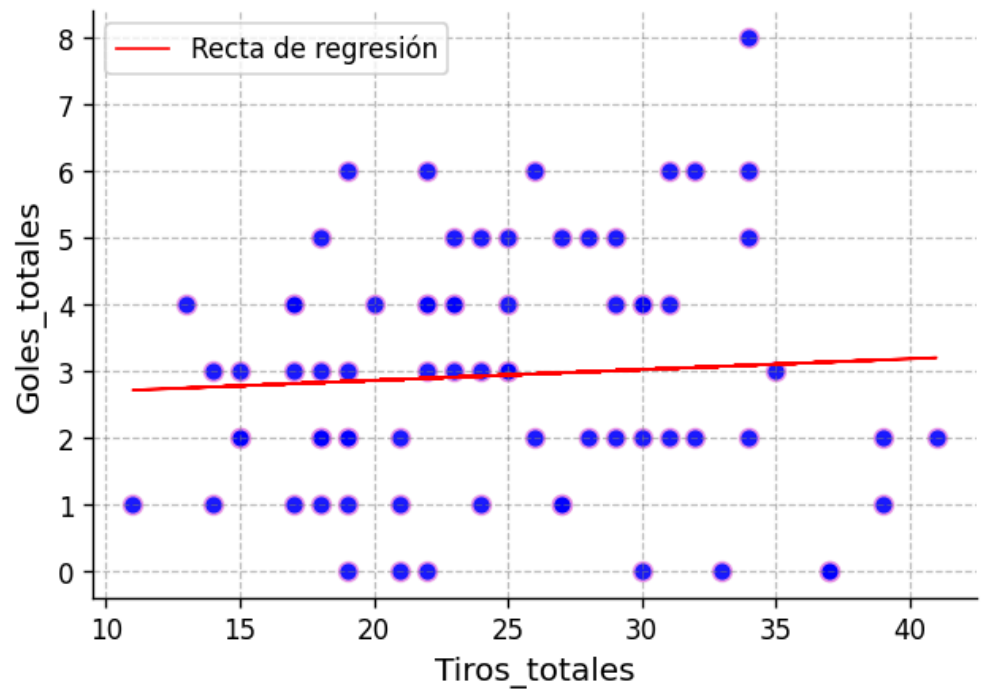
```

linestyle='--',
linewidth=0.7,
alpha=0.5,
color="gray"
)

plt.legend(
    loc='best',
    fontsize=10
)

```

<matplotlib.legend.Legend at 0x7d8200c59a30>



In []: `modelo.conf_int(alpha=0.05)`

	0	1
const	0.933014	4.128650
Tiros_totales	-0.046104	0.078514

CONCLUSIONES

1.-¿Se cumplió el objetivo del proyecto?

```

In [ ]: # Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4), # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120         # resolución del gráfico
)

# Calcular los residuales
residuales = modelo.resid

# Gráfico de dispersión
plt.scatter(
    X, residuales, # <-----
    marker="o",    # forma: googlear "matplotlib.markers"
    color='green', # color de los puntos
    edgecolor='black', # borde de los puntos
    alpha=0.5,     # transparencia: 0 es invisible y 1 es color
    sólido
    s=50,          # tamaño de los puntos
)

# Gráfico de línea
plt.plot(
    X, y_calculada * 0,
    color='red', # color de la línea
    linewidth=1.0, # grosor de la línea
    linestyle='-', # estilo de línea
    label='Recta de regresión'
)

# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Tiros_totales', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Residuales', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

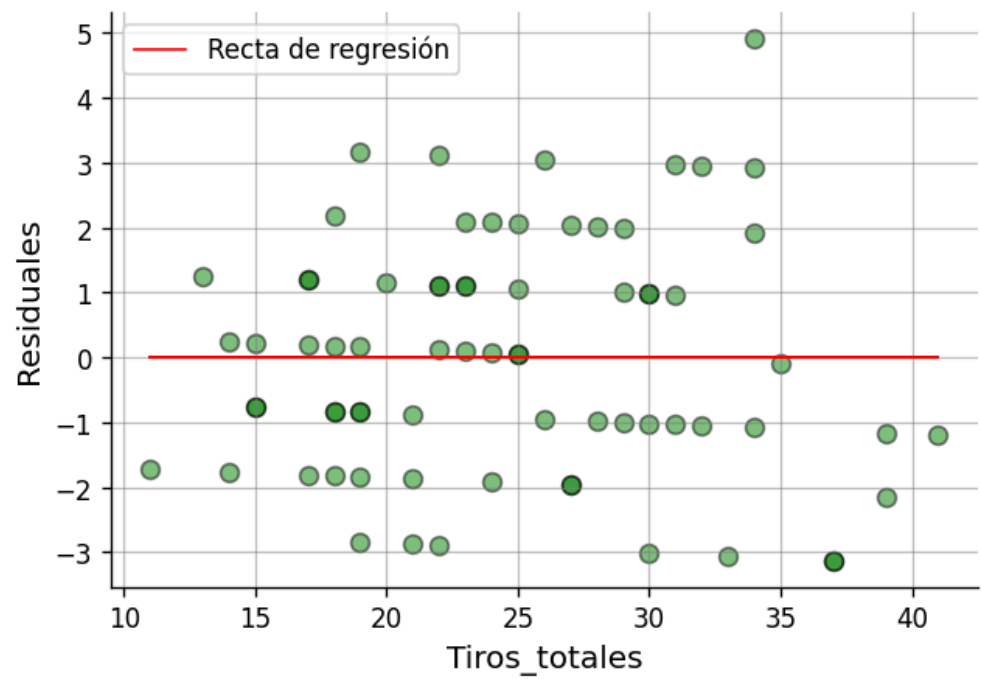
# Cuadrícula (opcional, pero didáctica)

```

```
plt.grid(
    visible=True,
    linestyle='-',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

plt.legend(
    fontsize=10,
    loc='best'
)
```

<matplotlib.legend.Legend at 0x7d81fe77f920>



```
In [ ]: from scipy.stats import shapiro
import matplotlib.pyplot as plt
import seaborn as sns

# n < 30, Shapiro-Wilk es el más confiable
# n >= 30, Histograma o Q-Q plot

# test de Shapiro-Wilk
1
# H0: Hay normalidad      0.4172971767713699
0.05
# H1: No hay normalidad
0

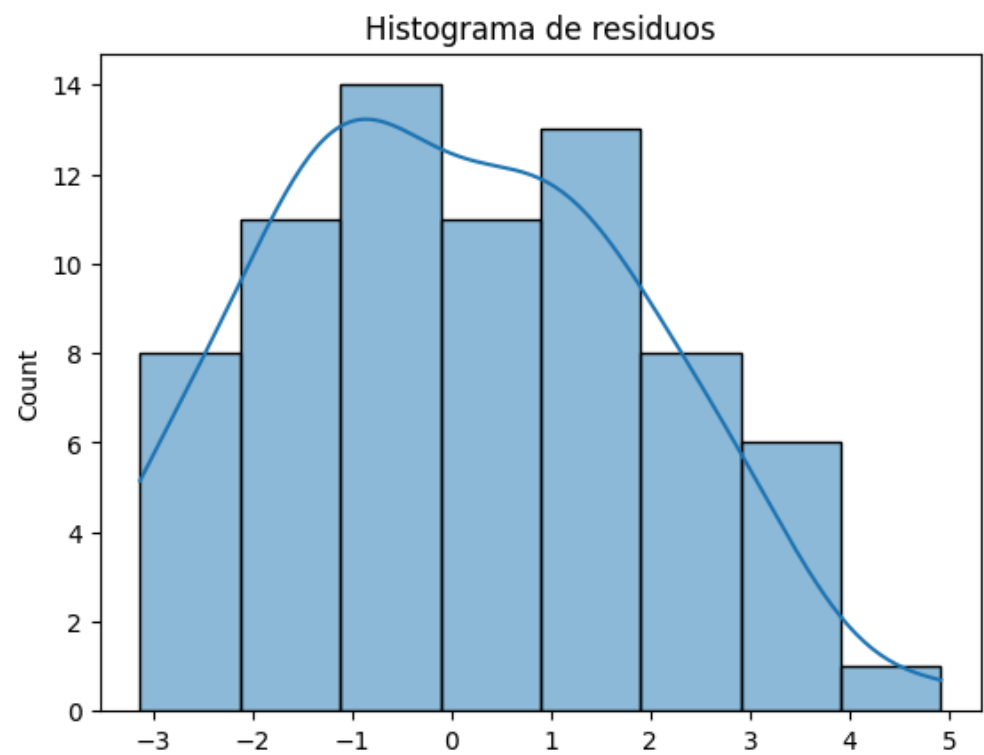
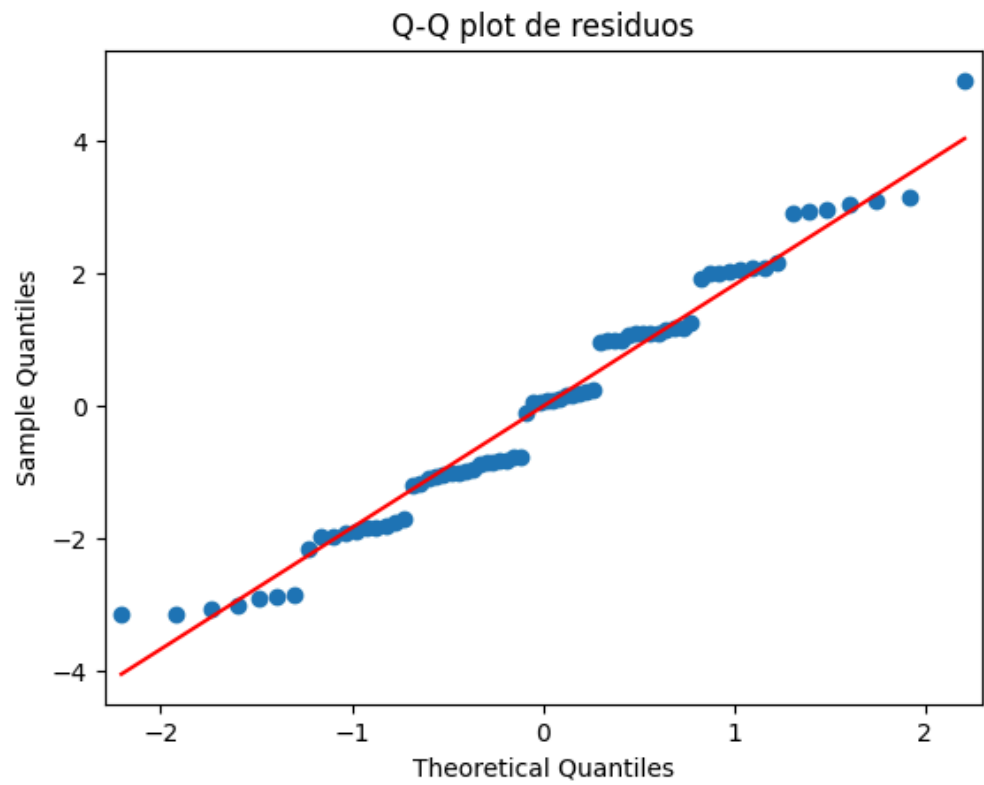
# Prueba de Shapiro-Wilk
stat, valor_p_sh = shapiro(residuales)
print(f"valor-p (Shapiro) = {valor_p_sh}")

# Visualización: Q-Q plot
sm.qqplot(residuales, line='s')
plt.title("Q-Q plot de residuos")
plt.show()

# Histograma
sns.histplot(residuales, kde=True)
plt.title("Histograma de residuos")
plt.show()# Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05

from statsmodels.stats.api import het_breuschpagan
import statsmodels.api as sm
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor-p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')
```

valor-p (Shapiro) = 0.08638047160895754



valor-p de Breusch-Pagan: 0.0111

```
In [ ]: # Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05

from statsmodels.stats.api import het_breuschpagan
import statsmodels.api as sm
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor-p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')
```

valor-p de Breusch-Pagan: 0.0111

```
In [ ]: import statsmodels.api as sm
from statsmodels.formula.api import ols

# Y ~ X
modelo_lineal = ols('Goles_totales ~ Tiros_totales', data=df).fit()
tabla_anova = sm.stats.anova_lm(modelo_lineal)
tabla_anova
```

	df	sum_sq	mean_sq	F	PR(>F)
Tiros_totales	1.0	0.929087	0.929087	0.269051	0.605605
Residual	70.0	241.723690	3.453196	NaN	NaN

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).

1. Metodología

- ¿Cómo se obtuvieron los datos? con una aplicación de fútbol que proporcionaba los datos necesarios
- ¿Qué población representa la muestra?
- ¿Cuál fue el tamaño de la muestra? 72 datos de mismos y diferentes días
- ¿Cuál es la variable independiente? Los goles totales
- ¿Cuál es la variable dependiente? Tiro de esquina
- ¿Cómo se midieron ambas variables?
- ¿Qué procedimiento estadístico se aplicó?
- ¿Por qué la regresión lineal simple es adecuada para este problema?

```
import pandas as pd
```

```
df = pd.read_csv("https://raw.githubusercontent.com/JazVargaskS20/EstadisticaVerano2026/refs/heads/main/StudentPe
```

```
df
```

	D'a_del_partido	Tiros_de_esquina	Tiros_totales	Goles_totales
0	11-jun	6	19	2
1	11-jun	9	22	3
2	12-jun	13	21	2
3	12-jun	4	25	5
4	13-jun	13	32	2
...	...	...	...	...
67	27-jun	5	14	3
68	27-jun	7	37	0
69	27-jun	6	22	4
70	27-jun	8	17	4
71	27-jun	3	22	6

72 rows × 4 columns

```
X = df['Tiros_de_esquina'] #Variable dependiente
Y = df['Goles_totales'] #Variable independiente
```

RESULTADOS

¿Qué muestran los gráficos de dispersión?

Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt

```
import matplotlib.pyplot as plt
```

Configuración general del gráfico

```
plt.figure(
    figsize=(6, 4), # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120 # resolución del gráfico
)
```

Gráfico de dispersión

```
plt.scatter(
    X, Y,
    marker="*", # forma: googlear "matplotlib.markers"
    color='#FD292F', # color de los puntos
    edgecolor='#E3BDFA', # borde de los puntos
    alpha=0.9, # transparencia: 0 es invisible y 1 es color sólido
    s=50, # tamaño de los puntos
)
```

Etiquetas de los ejes

eje x

```
plt.xlabel(
    'Tiros_de_esquina', # etiqueta del eje x
)
```

```

    fontsize=12 # tamaño de fuente
)

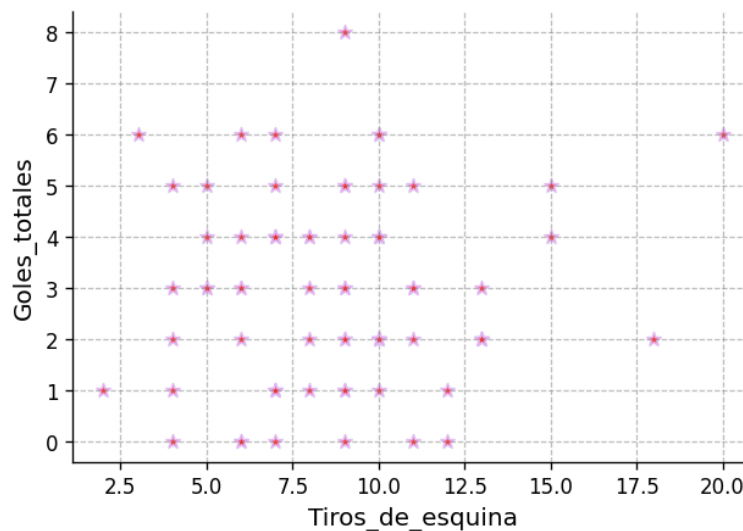
plt.ylabel(
    'Goles_totales', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Fuente de Los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False) # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

```



El gráfico de dispersión muestra los tiros totales (X) muestra los goles totales (Y)

Ademas que entre 2 a 20 tilos totales se pueden meter hasta 8 goles totales en un partido

2.-¿Cuál fue el valor del coeficiente de correlación y cómo se interpreta?

```

# Importar La función pearsonr desde scipy.stats
from scipy.stats import pearsonr

# Test de Pearson
# H0: rho = 0 (No hay correlación)
# H1: rho ≠ 0 (Sí hay correlación)
# alpha = 0.05

r, valor_p = pearsonr(X, Y)

print(f"Coefficiente de correlación: {r: 0.4f}")
print(f'valor_p: {valor_p: 0.4f}')

Coeficiente de correlación: 0.0543
valor_p: 0.6508

```

3.-¿Cuál fue el coeficiente de correlación y cómo se interpreta?

El coeficiente de correlacion es de $r=0.0543$ y esto indica que hay una linea muy baja

4.-¿Cuál fue el coeficiente de determinación y qué porcentaje de la variabilidad explica el modelo?

```

import statsmodels.api as sm
x_constante = sm.add_constant(X)

```

```

modelo = sm.OLS(Y, x_constante).fit()
y_calculada = modelo.predict(x_constante)

# Coeficiente de determinación
from sklearn.metrics import r2_score

r2 = r2_score(Y, y_calculada)

print(f"Coeficiente de determinación: {r2: 0.4%}")

Coeficiente de determinación: 0.2944%

```

5.-¿Cuál es la ecuación de regresión obtenida?

$$R^2 = 0.2944$$

6.-¿Qué significa la pendiente en el contexto del proyecto?

Una vez ajustado el modelo de regresión lineal, se obtiene un total de goles de $R^2 = 0.3829$. Entonces teniendo los tiros totales y el modelo ajustado, sólo podemos justificar la variabilidad en los tiros de esquina en un 38.29%, lo que es un poco bajo basándonos en que un partido hay más de 5 goles totales

7.-¿Qué indican los residuales sobre la calidad del modelo?

```

import pandas as pd
import statsmodels.api as sm

# Load the dataframe
df = pd.read_csv("https://raw.githubusercontent.com/JazVargasKS20/EstadisticaVerano2026/refs/heads/main/StudentPe")

# Define X and Y variables
Y = df["Goles_totales"] # Variable dependiente
X = df["Tiros_de_esquina"] # Variable independiente

# Add a constant to the independent variable
x_constante = sm.add_constant(X)

# Create and fit the OLS model
modelo = sm.OLS(Y, x_constante).fit()

# Print the model parameters
modelo.params

```

	0	
const	2.666679	
Tiros_de_esquina	0.030253	

dtype: float64

Indican que el constante es de 2.67 y los tiros totales es de 0.030

8.-¿Se cumplen los supuestos del modelo?

se podría decir que no dado que es muy bajo

9.-¿Qué predicción puede realizarse utilizando la ecuación obtenida?

La ecuación de la recta es:

$$\hat{y} = 2.666679 + 0.030253X$$

Este modelo estima que la constancia que fue: 2.666679 por lo que el valor esperado de goles totales en un partido es de 0 y que Goles totales es 0.030253 lo que indica que por tiro no hay tanta probabilidad que se meta en un partido

1.-¿Qué hallazgos principales se obtuvieron?

```

# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4), # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120 # resolución del gráfico
)

# Gráfico de dispersión
plt.scatter(

```

```

X, Y,
marker="*",          # forma: googlear "matplotlib.markers"
color='#E3BDFA',     # color de los puntos
edgecolor='#ED92E8',  # borde de los puntos
alpha=0.9,           # transparencia: 0 es invisible y 1 es color sólido
s=50,                # tamaño de los puntos
)

# Gráfico de Línea
plt.plot(
    X, y_calculada,
    color='#9792ED',  # color de la línea
    linewidth=1.0,     # grosor de la línea
    linestyle='--',    # estilo de línea
    label='Recta de regresión'
)

# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Tiros_de_esquina', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

# eje y
plt.ylabel(
    'Goles_totales', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

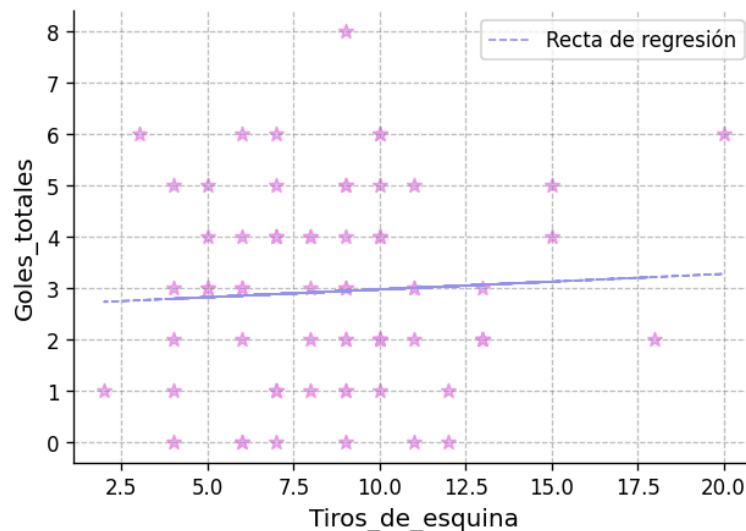
# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

plt.legend(
    loc='best',
    fontsize=10
)

```

<matplotlib.legend.Legend at 0x7af402b18680>



3.-¿Qué tan confiable ?

```
modelo.conf_int(alpha=0.05)
```

	0	1
const	1.429295	3.904062
Tiros_de_esquina	-0.102472	0.162978

CONCLUSIONES

1.-¿Se cumplió el objetivo del proyecto?

si, nos pudimos percatar que no hay tanta posibilidad de hacer un tiro de esquina y llegara a meter gol

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4), # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120         # resolución del gráfico
)

# Calcular los residuales
residuales = modelo.resid

# Gráfico de dispersión
plt.scatter(
    X, residuales, # <-----
    marker="o",    # forma: googlear "matplotlib.markers"
    color='#F52D3', # color de los puntos
    edgecolor='black', # borde de los puntos
    alpha=0.5,      # transparencia: 0 es invisible y 1 es color sólido
    s=50,           # tamaño de los puntos
)

# Gráfico de línea
plt.plot(
    X, y_calculada * 0,
    color='black', # color de la línea
    linewidth=1.0, # grosor de la línea
    linestyle='--', # estilo de línea
    label='Recta de regresión'
)

# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Tiros_de_esquina', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Residuales', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

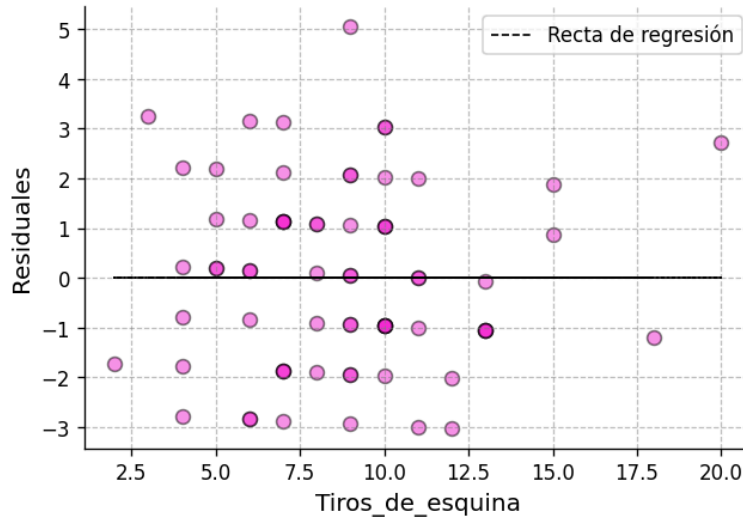
# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

plt.legend(
    fontsize=10,
```

```

loc='best'
)
<matplotlib.legend.Legend at 0x7af3fbeb8f0>

```



```

from scipy.stats import shapiro
import matplotlib.pyplot as plt
import seaborn as sns

# n < 30, Shapiro-Wilk es el más confiable
# n >= 30, Histograma o Q-Q plot

# test de Shapiro-Wilk
1
# H0: Hay normalidad    0.4172971767713699
0.05
# H1: No hay normalidad
0

# Prueba de Shapiro-Wilk
stat, valor_p_sh = shapiro(residuales)
print(f"valor-p (Shapiro) = {valor_p_sh}")

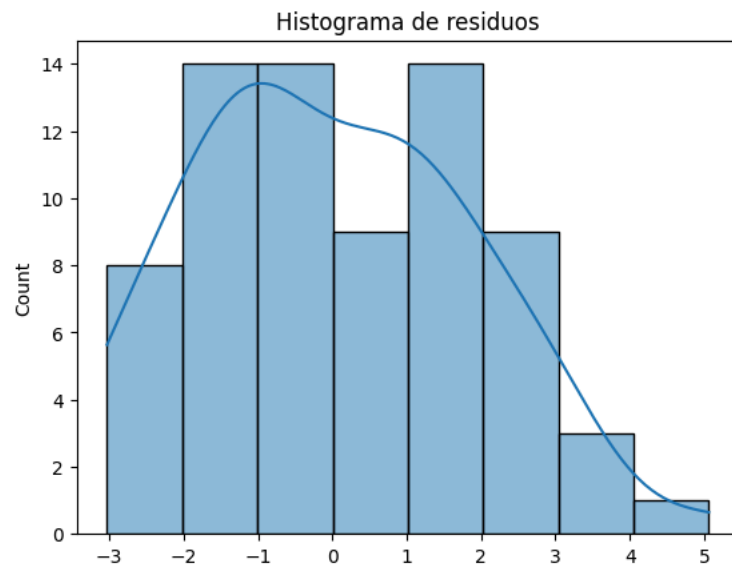
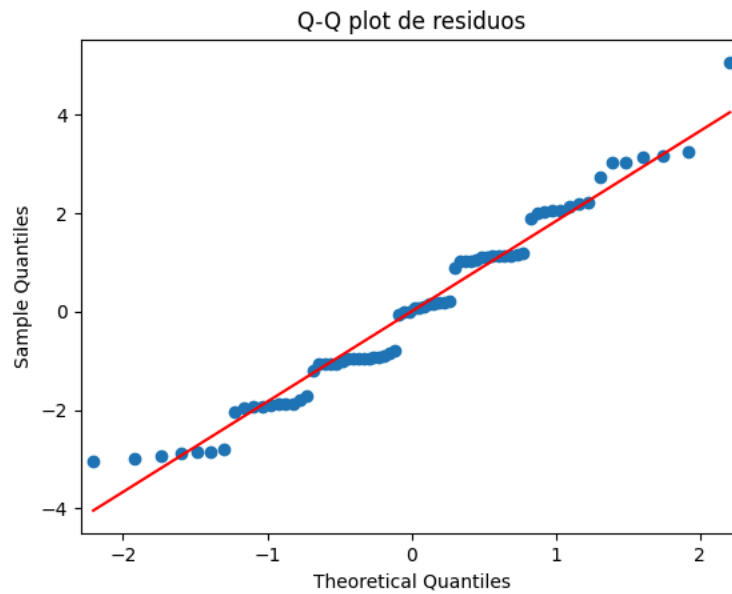
# Visualización: Q-Q plot
sm.qqplot(residuales, line='s')
plt.title("Q-Q plot de residuos")
plt.show()

# Histograma
sns.histplot(residuales, kde=True)
plt.title("Histograma de residuos")
plt.show()# Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05

from statsmodels.stats.api import het_breuschpagan
import statsmodels.api as sm
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor-p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')

valor_p (Shapiro) = 0.04312177616180001

```



valor-p de Breusch-Pagan: 0.6517

```
# Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05
```

```
from statsmodels.stats.api import het_breuschpagan
import statsmodels.api as sm
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor-p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')
```

valor-p de Breusch-Pagan: 0.6517

```
import statsmodels.api as sm
from statsmodels.formula.api import ols

# Y ~ X
modelo_lineal = ols('Goles_totales ~ Tiros_de_esquina', data=df).fit()
tabla_anova = sm.stats.anova_lm(modelo_lineal)
tabla_anova
```

	df	sum_sq	mean_sq	F	PR(>F)
Tiros_de_esquina	1.0	0.714317	0.714317	0.206673	0.650794
Residual	70.0	241.938461	3.456264	NaN	NaN

Conclusion

si se cumplio con el objetivo ya que se pudo mostrar que los partidos donde hubo mas tiros de balon y a su vez goles son los que tienen plantillas o equipos mas estables y seguros para poder ganar este campeonato, se aprendio a analizar mediante graficos las posibles variantes del problema a detectar y tambien diferenciar a la variable dependiente de la indebandiente, fue de gran utilidad dicho modelo como ya se menciona para poder plasmar de una manera mas clara y sencilla de entender, las principales limitaciones fueron encontrar tantos datos esparcidos en la app.

